

별첨3

프로젝트 결과 요약보고서

프로젝트명	TraceInspector - 추적 가능한 시각화 기반 코드 정적분석기		
협력기관(국가)		지도교수님	김형균 교수
수행기간	2025. 03. 02 ~ 2025. 05. 31 (3개월)		
참여인원	구분	인원수	성명(모두 기재)
	교수	1	김형균 교수
	학부생	2	김신영, 강민성
	기업체 멘토 1	1	강동호(LG 유플러스 볼트업)
	기업체 멘토 2	1	이광현(SK 하이닉스)
	계	5	
추진배경	코드 정적 분석은 프로그램을 실행하지 않고 코드를 논리적으로 분석하여 동적 특성을 파악하는 기법이며 런타임 오류, 주어진 명세 준수 등 다양한 논리적/의미적 성질을 검증하기 위해 사용된다. 이러한 정적 분석 기술은 IDE와 CI 프로세스 등 다양한 개발 도구에 탑재되어 있으며, 안전한 소프트웨어를 개발하기 위해 적극 활용되고 있다. 기존 정적 분석 도구들은 대규모 소프트웨어 프로젝트의 수많은 소스 파일들에 대해 다양한 오류들을 신속하고, 정확하게 탐지하는 것을 핵심 목표로 한다. IDE에서는 소스가 수정될 때마다, CI에서는 새로운 commit이 등록될 때마다 분석기가 동작해야 하기 때문이다. 하지만 이런 편리한 정적 분석기가 답해주지 못하는 질문들이 있다: 어떤 오류들이 어디서 발생한 것이 아니라, 어떻게 오류들이 탐지되었는지에 대해서는 구체적인 정보를 제공하지 않는다. 이에 따라 분석기가 어떤 추론 과정을 통해 분석을 수행했는지, 그리고 그런 추론이 논리적으로 유효한지에 대해서는 사용자가 파악하기 매우 어렵다는 한계가 존재한다. TraceInspector는 단순히 오류를 탐지해서 알려주는 정적 분석기를 넘어, 분석 과정을 시각적으로 나타내어 사용자가 어떤 과정을 통해 오류가 탐지되었는지 해석 가능하게끔 구현된 정적 분석기이다. 즉 분석기의 동작 흐름을 사용자가 직접 관찰하며 올바른 추론을 분석기가 수행했는지, 그리고 어떻게 정적 분석이 이루어졌는지를 전달하고자 본 프로젝트를 구상하였다. 결론적으로 TraceInspector을 통해 기대하고자 하는 효과는 다음과 같다: 다양한 오류를 신속하게 분석하는게 아닌, 어떤 과정으로 정적 분석이 프로그램을 처리하고 오류를 탐지하는지에 대한 정보 전달 및 시각화		

- 정적 분석 알고리즘 및 관련 이론에 대한 **교육적 자료**로 활용

배경 및 내용

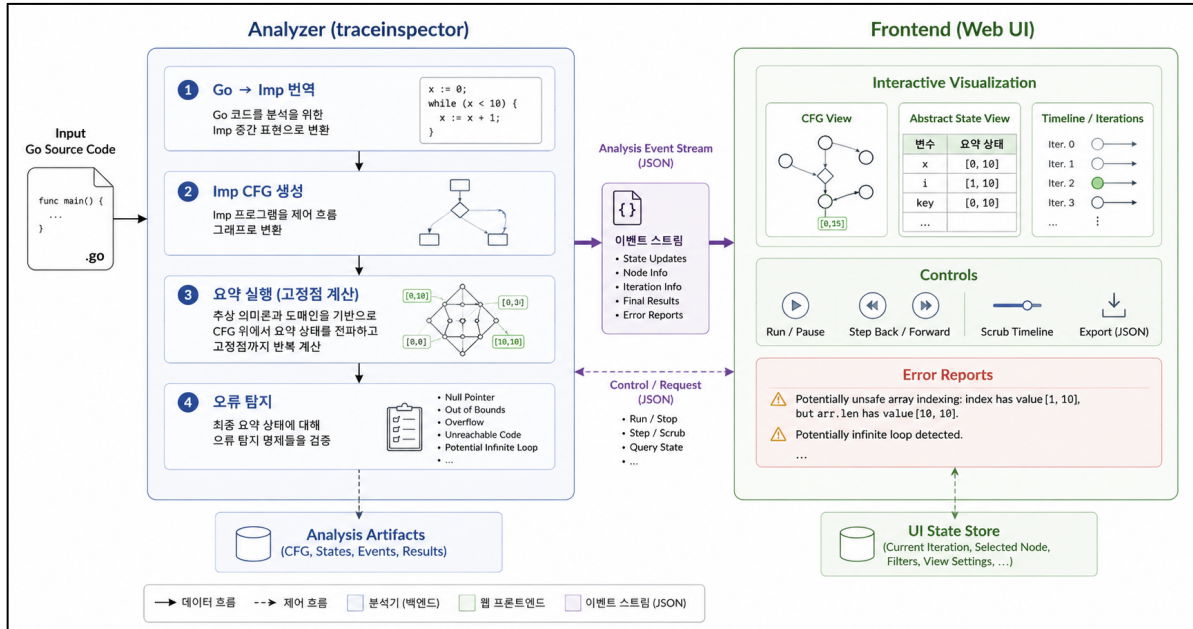


그림 1: TracInspector 전체 시스템 구성도

TracInspector는 정적 분석을 수행하는 분석기(TracInspector Analyzer)와 분석 결과를 시각적으로 보여주는 웹 인터페이스(TracInspector Frontend)로 구성되어 있다.

분석기(TracInspector Analyzer)

분석기는 Go 언어로 작성된 단일 실행 파일이며, 입력으로 임의의 Go 소스 파일을 받아 분석 결과를 출력한다. 분석기는 잠재적인 오류가 탐지되었을 경우 오류에 대한 정보와 위치에 대해 출력하게 된다.

분석기는 요약해석(abstract interpretation)이론을 바탕으로 구현되었다. 요약해석이란 실제 프로그램 실행 결과를 '요약 도메인'이라는 수학적 구조를 통해 나타내어 근사하는 기법이다.

요약해석의 핵심적인 특징이자 강점은 논리적으로 건전(sound)함이 보장된다는 것이다. 즉 요약해석을 통해 근사한 프로그램 상태에서 오류가 발생하지 않음이 판별되면, 실제로 **실행 가능한 모든 상태**에 대해 오류가 발생하지 않음이 수학적으로 보장된다. 따라서 TracInspector 분석기가 가정한 실행 의미론 및 분석 모형 내에서 특정 오류를 탐지하지 못했을 경우, 실제로 실행했을 때 **어떠한 경우에도 해당 오류가 발생하지 않을 것임**이 보장된다.

이러한 분석을 가능케 위해 분석기는 입력 Go 소스에 대해 다음의 4단계를 수행한다:

1. 입력 Go 소스를 Imp 중간 표현 언어로 번역
2. Imp 소스에 대한 제어 흐름 그래프(control-flow graph, CFG)로 생성

3. CFG 위에서 요약해석(abstract interpretation) 이론을 바탕으로 프로그램 상태를 요약 근사
4. 요약 프로그램 상태에 대해 오류 상태 여부를 확인

가장 먼저 입력받은 Go 소스 파일을 파싱 후 Imp라는 중간 표현 언어로 번역한다. 분석기의 정적 분석 로직은 Go 언어에 대해서 동작하는 것이 아니라 Imp 언어의 요소들에 대해 동작한다. 이러한 번역을 수행하는 이유는 크게 두가지로 볼 수 있다:

1. Go AST의 구문적인 요소를 소거하여 오로지 논리적인 흐름만 간결하게 표현할 수 있는 형태로 정제하기 위해.
2. 향후 타 언어를 지원하기 위해서는 분석 로직 전체를 새로운 언어에 이식할 필요 없이 새 언어에서 Imp로의 번역기만을 구현하면 완성.

Imp 언어는 간단한 명령형 언어로 설계되어 있다. 정수, Boolean, 동적 배열 등 기본적인 타입과 더불어 조건문과 while문 등 간단한 제어 흐름과 함수를 지원한다. 이렇게 정의된 Imp언어는 Go AST보다 실제 프로그램 논리와 더 근접하고 깔끔한 형태로 표현 가능케 하고 향후 변환 및 분석 단계 구현을 훨씬 수월하게 만들어준다.

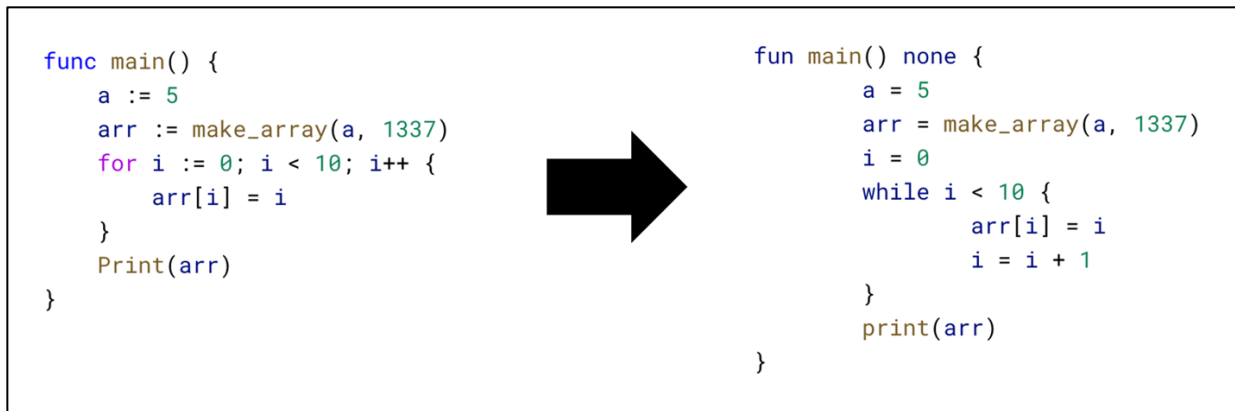


그림 2: 예시 Go 코드와 Imp로 번역한 결과

Imp 코드로 나타낸 후 다음 단계는 코드 형태에서 알고리즘적으로 다룰 수 있는 제어 흐름 그래프(control-flow graph, CFG)로 변환하는 작업이다. CFG는 컴파일러 등에서 필수적으로 활용하는 자료 구조로 주어진 코드의 실행 흐름을 그래프로 나타낸 형식이다. 표현 방식은 용도에 따라 조금씩 다를 수 있지만 공통적으로 개별 명령문들을 각각의 정점으로, 후속 명령 및 조건문에 따른 분기를 간선으로 표현하는 directed graph로 나타낸다.

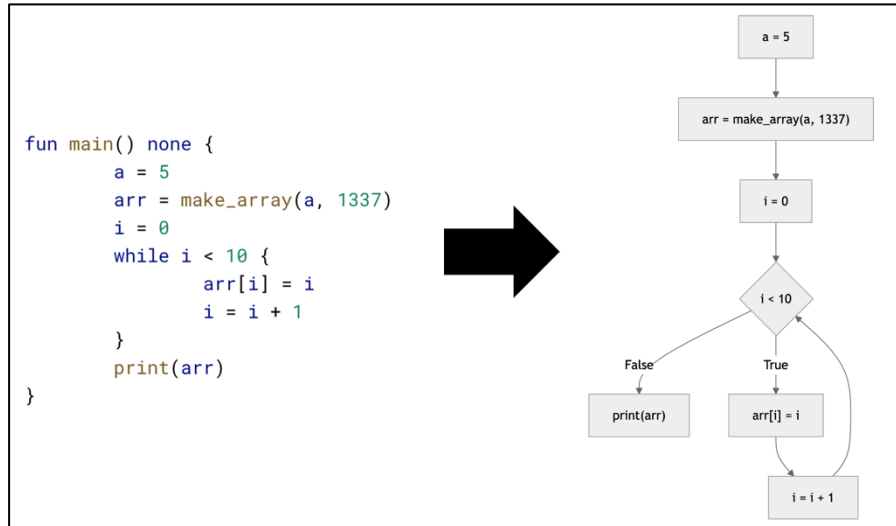


그림 3: 예시 Imp 코드를 CFG로 변환한 모습

입력 코드를 CFG로 나타낸 이후, 분석기는 Imp에 대해 정의된 요약 의미론을 바탕으로 CFG의 각 정점마다 요약 프로그램 상태를 계산한다. 계산된 요약 프로그램 상태는 실제 프로그램 실행 시 해당 코드가 실행될 메모리 상태를 근사한 결과이다.

현재 구현된 분석기는 정수값의 경우 ‘구간 산술 요약도메인’이라는 구조를 통해 근사한다. 구체적인 정수값을 계산하는 것이 아니라 각 변수와 수식들의 최소값과 최대값만을 구하고, 이를 전파하면서 각 프로그램 상태에서 존재 가능한 값을 계산하는 방식이다.

그림 4를 예시로 보면, while loop 내부에서 실제 계산시 존재 가능한 i 의 값은 $\{0, 1, 2, \dots, 8, 9\}$ 이지만 분석기는 이러한 정보를 $[0, 9]$ 라는 최소/최대값만으로 표현하여 계산한다. 분석의 논리적 건전성을 위해서는 요약 정보가 실제로 존재하지 않을 값들을 반영해도 괜찮지만, 존재 가능한 값들은 빠짐없이 반영되어야 한다.

Boolean, 배열의 경우에 대해서도 각각에 대응되는 요약도메인 구조를 활용하여 근사를 취하게 된다.

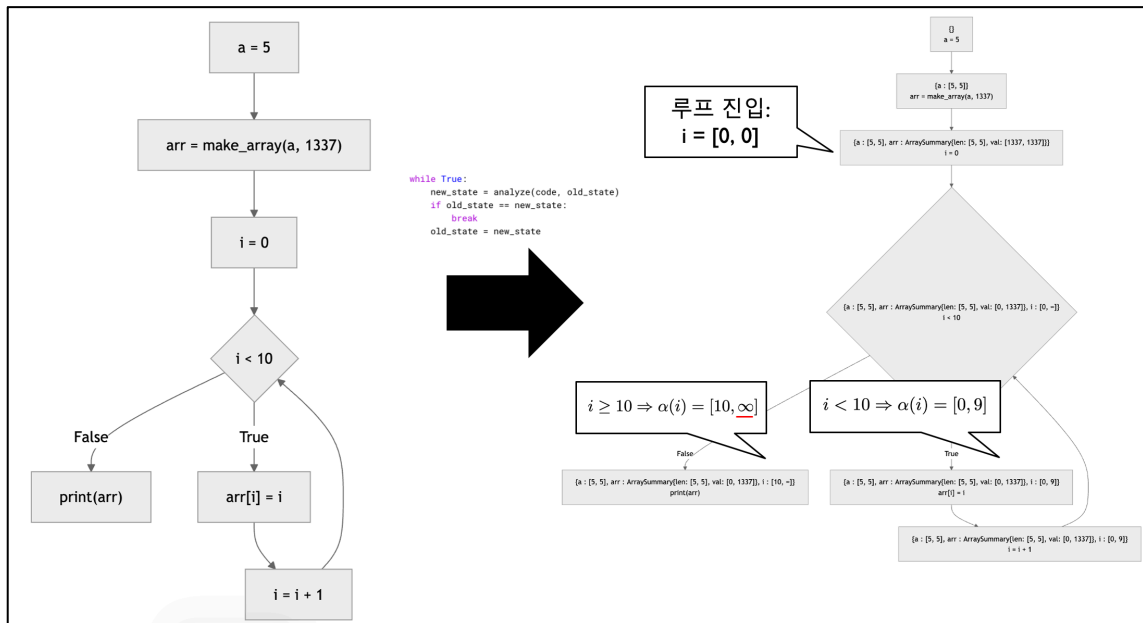


그림 5: 구간 산술도메인을 이용한 루프 진입 및 탈출 상태 근사

이러한 요약 상태 계산은 한번의 과정을 통해 완성되는 것이 아니라, 아무 정보가 없는 초기 상태에서부터 CFG를 계속해서 탐색하며 정보를 보완하고 갱신하는 과정을 반복하면서 구체화되고 분석이 정교해진다. 이러한 과정은 더 이상 새로운 정보가 발생하지 않는 고정점(fixed point)에 도달할 때까지 반복된다. TracelInspector는 이러한 정보의 계산 및 갱신 과정을 단일 결과로 나타내지 않고 **변화의 흐름**으로 온전히 사용자에게 나타내고자 하였다.

요약 상태에 대한 고정점이 계산된 후, 분석기가 탐지하고자 하는 오류들이 각 정점에서 발생 가능한지 판별한다. 오류별 존재 여부를 판별하는 논리적 성질을 상태마다 계산하여 탐지된 오류의 유형과 위치를 반환한다. 현재 TracelInspector가 확인하는 오류는 다음과 같다:

- 배열 인덱싱 범위 오류: 주어진 배열에 대해 인덱스 수식이 유효한 범위 내의 값인지 검증
- 조건문 항진/모순명제 여부 판별: 주어진 조건문이 모든 입력에 대해 항상 참이거나 거짓인지 판별. 이에 따라 도달 불가능한 분기문과 코드를 파악.

Type	Function	Line	Message
	insertionSort	15	Potentially unsafe array indexing: index expr 'j + 1' has value [1, 10], but 'arr.Len' has value [10, 10].
	insertionSort	23	Potentially unsafe array indexing: index expr 'j + 1' has value [0, 10], but 'arr.Len' has value [10, 10].
	main	25	Dead branch: boolean expression 'a < b' always evaluates to false
	main	15	Dead branch: boolean expression 'a < b' always evaluates to true

그림 6: TracelInspector Frontend에 표시된 오류 정보 예시

최종적으로 분석기가 계산한 단계별 요약 상태와 오류 정보는 TracelInspector Frontend에 전달되며, 이 정보는 사용자에게 시각적 형태로 표시된다.

웹 인터페이스(TracelInspector Frontend)

TracelInspector Frontend는 NodeJS, vite, React를 활용하여 구현되었으며, CFG, 분석기의 요약 상태 계산 흐름과 오류 판별 결과를 보여주는 구조로 디자인되었다.

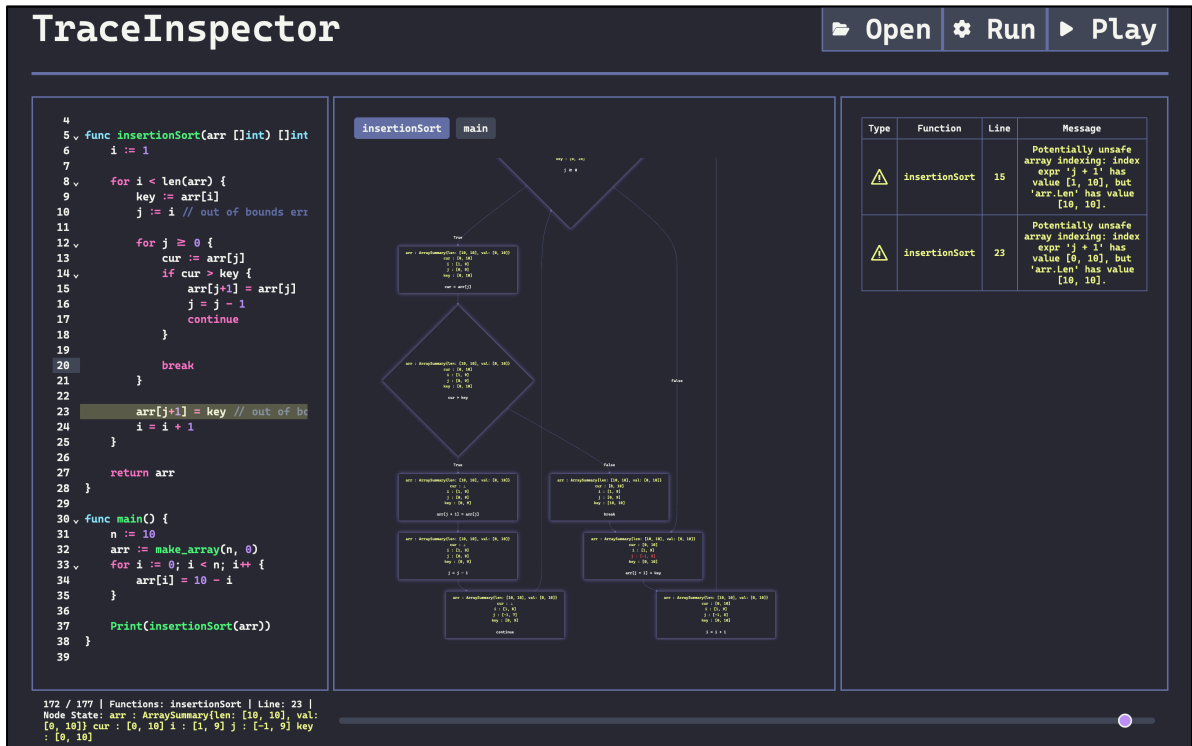


그림 7: Tracelnspector Frontend 인터페이스

- 좌측: 입력 코드 및 현재 요약 상태에 대응되는 코드 표시
- 우측: 탐지된 오류 정보
- 중앙: 함수별 전체 CFG 및 누적 요약 상태
- 하단: 요약 상태 고정점 계산 단계 탐색 슬라이더

TraceInspector의 핵심 기능은 요약 상태 계산 과정을 시각적으로 보여주는 것이다. 이에 따라 인터페이스 하단의 슬라이더를 통해 아무 정보가 주어지지 않은 시작 상태에서부터 최종 고정점 상태까지 어떤 흐름에 따라 정보가 계산되고 갱신되는지 직접 흐름을 탐색하고 파악할 수 있다. 더불어 오류가 발생한 경우 해당 오류를 클릭 시 오류가 탐지된 상태로 이동되어, 어떤 근거로 인해 오류로 판별되었는지 바로 파악이 가능하다.



그림 8: 슬라이더를 활용하여 요약 상태 계산 과정을 탐색하는 모습

TraceInspector - 추적 가능한 시각화 기반 코드 정적분석기

배경

정적 분석(Static Analysis)은 프로그램을 실행하지 않고 코드의 실행 특성을 추론하여 잠재적인 오류를 탐지하는 기술이다. IDE, 컴파일러, CI 등 다양한 개발 과정에서 코드 품질과 안정성을 높이기 위해 활용되고 있음.

기존 분석기들은 분석 결과만 제시할 뿐 어떻게 오류가 탐지되었는지 파악하기 어렵고, 분석 과정을 일종의 “블랙박스”처럼 받아들이고 결과를 맹신하게 됨.

TraceInspector 소개

TraceInspector는 요약해석(Abtract Interpretation) 이론을 바탕으로 한 정적 분석기와 더불어 분석 과정을 시각적으로 보여주는 인터페이스로 구성되어 있음. 분석 중 생성되는 요약 상태(Abtract State)의 변화 과정을 사용자에게 실시간으로 보여주어 각 프로그램 지점에서 정보가 어떻게 전파 및 정제되는지를 직관적으로 관찰할 수 있도록 구현함.

분석 과정

1: Go 코드 → 중간 표현 언어(Imp)

Go AST를 간단한 형태의 Imp 언어로 번역. 분석기는 Imp 코드에 대해 동작하며, 분석기 구현 및 타 언어 이식 시 용이하도록 설계.

```
func main() {  
  a := 5  
  arr := make_array(a, 1027)  
  for i := 0; i < 10; i++ {  
    arr[i] = i  
  }  
  Print(arr)  
}
```



```
fun main() none {  
  a = 5  
  arr = make_array(a, 1027)  
  i = 0  
  while i < 10 {  
    arr[i] = i  
    i = i + 1  
  }  
  print(arr)  
}
```

2: Imp → 제어 흐름 그래프(CFG)

프로그램 내 분기, 반복 등의 제어 흐름 단위로 나타내는 제어 흐름 그래프 형태로 표현. 각각의 명령문들은 단일 노드로 표현됨.



3: CFG 위에서 고정점 근사

CFG 노드별 진입 시점에서의 프로그램 상태를 근사. 고정점에 도달할 때까지 반복적으로 요약실행 및 상태 join/widening 수행.



분석 예시: 무한 루프 및 Dead Code Detection

```
1 func main() {  
2   a := 1  
3   for a > 0 {  
4     a++  
5   }  
6   Print(a)  
7 }
```

무한히 반복되는 코드의 경우 계산된 상태에 대해 widening 기법을 적용하여 분석이 종료할 수 있도록 유도. 이후 filtering을 통해 결과를 정제하여 정확도 손실을 방지. 추가로 항진/모순명제를 탐지하여 도달할 수 없는 코드에 대해 정보.

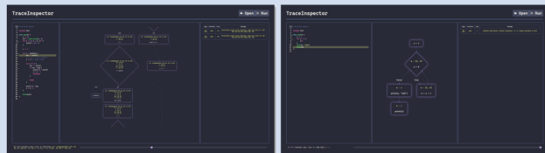
분석 예시: 배열 인덱싱 범위

```
1 i := 1  
2 for i < len(arr) {  
3   key := arr[i]  
4   arr[i+1] = arr[i]  
5 Print(i)
```

i: [1, 1]
i: [1, 10], len(arr): [10, 10]
i: [1, 9]
i: [1, 9], key: [0, 10]
i: [10, 10], key: [0, 10]

배열의 길이와 인덱스 변수에 대한 요약상태를 계산 후 안전한 인덱싱 범위인지 확인.

시각화 및 인터페이스



- 분석 과정에서 생성되는 상태 변화를 단계별로 재생 가능하도록 구현.
- 함수별 및 단계별 상태를 자유롭게 탐색 가능.

한계 및 향후 계획

- 관계형 요약도메인 : $i < \text{len}(\text{arr})$ 등과 같이 구체적인 값이 아닌 변수간의 논리적 관계를 나타낼 수 있는 대수적 도메인.
- 분석 정확도 개선 : reduced product domain, narrowing 구현 및 SimpleProp 체계 확장.
- Pointer inference : 포인터 및 메모리 동작 추상화를 통한 포인터 연산 검증.
- C → Imp, Rust → Imp 등 Imp를 활용한 다양한 언어 지원.
- 보다 다양한 언어 기능 지원 : 확장된 타입 체계 등.

2조 - 20212978 김신영, 20212952 강민성
국민대학교 소프트웨어학부



기대효과

TraceInspector는 기존 정적 분석기들처럼 대규모 소프트웨어에 대한 강력한 분석 성능과 다양한 기능 지원을 목표로 설계된 도구가 아니다. 현재는 Go언어의 일부 기능과 오류 유형을 제한적으로 탐지하기에, 실무 환경에서 유용한 범용적 도구로 보기에는 한계가 있다.

반면 TraceInspector는 현존하는 정적 분석기들처럼 추론 과정을 사용자로부터 숨기는 프로그램으로 동작하지 않고, 분석 과정 자체를 사용자가 관찰하고 해석할 수 있도록 설계되었다. 프로그래밍 경험이 풍부하고 정적 분석기를 적극적으로 활용하는 숙련된 프로그래머더라도 정적 분석이 어떠한 원리로 동작하는지에 대해서 잘 아는 경우는 드물다. TraceInspector는 이러한 분석 과정에 대한 이해를 돕고 정적 분석 도구를 보다 잘 활용할 수 있도록 교육적 역할을 수행하기를 기대한다.

마지막으로 단순히 프로그램을 작성하는 것을 넘어, 프로그램 자체를 계산과 논증의 대상으로 바라보는 관점을 소개하고자 하였다. 정적 분석은 논리적 결정가능성(decidability), 튜링의 정지 문제, Rice's Theorem 등 프로그래밍 언어론과 계산 이론의 근본적인 주제들과 밀접한 관련성을 가지고 있다. 우리가 작성하는 프로그램의 의미와 동작에 대해 심층적으로 바라볼 수 있는 계기를 제공하고, 이를 바탕으로 보다 안전하고 논리적인 소프트웨어 개발에 대한 관심을 유도할 수 있기를 기대한다.

결과

TraceInspector는 처음 실행 시 그림 9와 같은 인터페이스로 구성되어 있다:

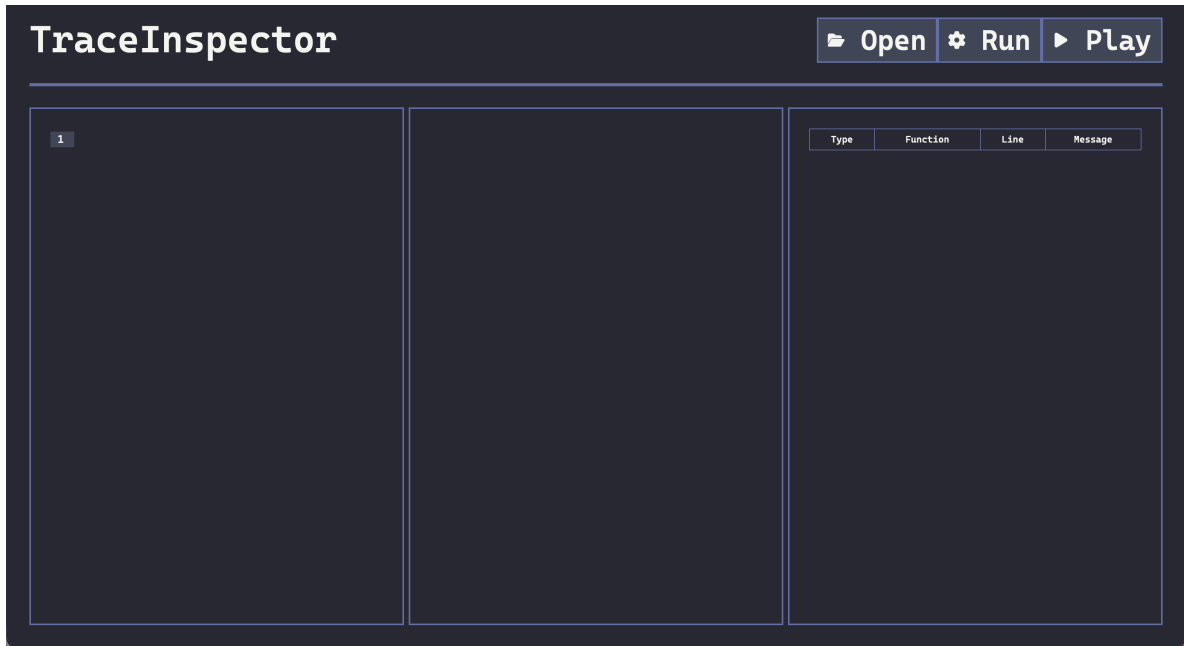


그림 9: 초기 상태의 TraceInspector 인터페이스

이후 Open을 통해 .go 파일을 업로드 후, Run을 눌러 분석을 실행할 수 있다.

TraceInspector

Open Run Play

```
1 //go:build ignore
2
3 package main
4
5 func insertionSort(arr []int) []int {
6     i := 1
7
8     for i < len(arr) {
9         key := arr[i]
10        j := i - 1
11
12        for j >= 0 {
13            cur := arr[j]
14            if cur > key {
15                arr[j+1] = arr[j]
16                j = j - 1
17                continue
18            }
19            break
20        }
21
22        arr[j+1] = key
23        i = i + 1
24    }
25
26    return arr
27 }
28
29 func main() {
30     n := 10
31     arr := make_array(n, 0)
32     for i := 0; i < n; i++ {
33         arr[i] = 10 - i
34     }
35 }
36
```

insertionSort main

Type	Message
✓	No warnings or errors found!

1 / 171 | Functions: main | Line: 31 | Node State: None

그림 10: 분석 후 오류가 탐지되지 않은 경우

공통적으로 분석 후 좌측에 소스 코드, 중앙에 CFG, 우측에 오류 정보가 표시된다. 그림 10과 같이 오류가 탐지되지 않았을 경우 오류의 부재를 표시한다.

TraceInspector

Open Run Play

```
1 //go:build ignore
2
3 package main
4
5 func insertionSort(arr []int) []int {
6     i := 1
7
8     for i < len(arr) {
9         key := arr[i]
10        j := i // out of bounds error
11
12        for j >= 0 {
13            cur := arr[j]
14            if cur > key {
15                arr[j+1] = arr[j]
16                j = j - 1
17                continue
18            }
19            break
20        }
21
22        arr[j+1] = key // out of
23        i = i + 1
24    }
25
26    return arr
27 }
28
29 func main() {
30     n := 10
31     arr := make_array(n, 0)
32     for i := 0; i < n; i++ {
33         arr[i] = 10 - i
34     }
35 }
36
```

insertionSort main

Type	Function	Line	Message
⚠	insertionSort	15	Potentially unsafe array indexing: index expr 'j + 1' has value [1, 10], but 'arr.Len' has value [10, 10].
⚠	insertionSort	23	Potentially unsafe array indexing: index expr 'j + 1' has value [0, 10], but 'arr.Len' has value [10, 10].

172 / 177 | Functions: insertionSort | Line: 23 | Node State: arr : ArraySummary{len: [10, 10], val: [0, 10]} cur : [0, 10] i : [1, 9] j : [-1, 9] key : [0, 10]

그림 11: 오류가 탐지된 프로그램에 대해 상태 탐색을 통한 과정 파악

반면 그림 12와 같이 오류가 탐지되었을 경우 슬라이더 하단의 버튼을 통해 각 오류별 탐지 상태

로 이동할 수 있고, 전후 상태를 슬라이더를 통해 직접 확인하며 어떻게 오류가 탐지되었는지 파악할 수 있다. CFG 그래프 영역의 경우 자유롭게 이동 및 확대가 가능하다.

멘토링의 경우 두 차례에 걸쳐 진행되었으며 정적 분석기가 현업에서 활용되고 있는 방향, TraceInspector 프로젝트 주제 소개 및 구현 방법 소개, 효율적인 정보 전달을 위한 인터페이스 개선 피드백과 관련된 조언을 얻을 수 있었다.

